

# Exercise 7: An Agent Team Builds Your Frontend

**Session:** 7 — Agentic Workflows **Duration:** 35 minutes, plus a 5-minute closing round **Project:** `bookstore-web`, the frontend you specified in Exercise 6.

## Goal

---

In Exercise 6 you wrote a spec, a plan and a task list, and no code. In this exercise an agent team builds the code. You do four things:

1. Build the shared part of the frontend, in one Claude session.
2. Write one prompt that gives each user story to its own teammate, and improve that prompt with a coach.
3. Start the team and watch it work.
4. Check the result against the constitution and your spec.

You will not write application code yourself.

## Before you start

---

Titles in *italics* are slide titles from this session.

**The words in this sheet.** They come from the slide *Agent Teams: Multi-Agent Coordination*. The **lead** is the Claude Code session you type in. It starts the teammates, hands out tasks and collects the results. A **teammate** is a separate Claude Code session, started by the lead. It has its own context window and its own task. You can open its transcript and talk to it directly. The **agent team** is the lead plus its teammates. They share one task list.

**No training mode here.** `bookstore-web` has no `CLAUDE.md`. Claude answers directly in this project, the same as in Exercise 6.

**Agent teams are experimental.** They are off unless a flag is set. This project sets the flag in `bookstore-web/.claude/settings.json`. You checked that during preparation. If no team starts in task 3, tell the trainer. Do not spend your time on it.

**Do you have a spec?** This exercise needs the `specs/001-*/` folder you committed at the end of Exercise 6. Check:

```
cd ../bookstore-web && ls specs/
```

No folder, or an unfinished one? Use the reference spec instead. It was written for this exercise and has the same format:

```
cp -r specs-reference/001-browse-books specs/  
git add specs/ && git commit -m "spec: reference spec for exercise 7"
```

**Three terminals**, the same as in Exercise 6.

**Terminal 1, your backend**, from the `bookstore-kt` project directory:

```
./gradlew run
```

**Terminal 2, Claude Code, started from** `bookstore-web` :

```
cd ../bookstore-web && claude
```

This session is your lead for the whole exercise. Keep it open.

**Terminal 3, a plain shell**, for `npm run dev`, `curl` and `git`. Task 1 tells you when to move it into `bookstore-web`.

Keep a browser tab ready for `http://localhost:5173`. It shows an empty page until task 1 is done.

---

## Tasks

---

### 1. Build the foundation, one session (6 min)

You produce the **foundation**: Phase 1 and Phase 2 of `tasks.md`. These are the files every story needs, such as the API client, the page shell and the stylesheet (*How the Exercise Works*). The constitution says no story may start before the foundation is done. So this part is not parallel work. One session, the lead, does it.

1. Open `specs/001-*/tasks.md` and find **Phase 1** and **Phase 2**. Write down the files those phases create, in terminal 3 or on paper. Keep the list. Task 2 needs it.

2. Send the lead this prompt. Copy it as written:

Implement Phase 1 and Phase 2 of `specs/001-*/tasks.md`, and nothing from any user story. Follow `.specify/memory/constitution.md`. Tick each task off in `tasks.md` when it is done. When you finish, list the files you created.

The run takes two to four minutes. **Do not wait. Start task 2 now.** Drafting a prompt changes no files, so it cannot disturb this run.

3. When the lead reports back, start the frontend in terminal 3:

```
cd ../bookstore-web && npm run dev
```

Open `http://localhost:5173`. You should see the page shell, and no errors in the browser console.

4. Commit and tag. The tag marks where the foundation ends and story work begins. The check in task 4 uses it:

```
git add -A && git commit -m "foundation" && git tag foundation
```

**Done when:** Phase 1 and 2 are ticked in `tasks.md`, the page shell loads on port 5173, and the foundation is committed.

## 2. Draft the team prompt (9 min)

You produce one prompt that turns the lead into a team lead. This is the core of the exercise.

`tasks.md` already says how to split the work between teammates. Find the section **Parallel Team Strategy** near the bottom. Your prompt turns that plan into instructions the lead can follow.

A teammate starts with its own, empty context. It does not see this chat. Everything it must know is in your prompt, or in a file your prompt names.

Each user story in `tasks.md` has an **Independent Test**: the steps from `quickstart.md` that must pass for that story. The worked example below uses it.

Here is a worked example from another project. Copy the shape, not the content. It is not complete. The coach will tell you what is missing.

Read `specs/002-*/tasks.md` . Spawn an agent team with one teammate per user story, named after the story: `us1-search` , `us2-export` . Each teammate claims only the tasks tagged with its story. Each teammate is done when its story's Independent Test in `tasks.md` passes against the running backend and its tasks are ticked. Report per story: files changed, and the result of its Independent Test.

The coach is the `/parallel-coach 2` command. It reads your draft and points out gaps. It never writes the prompt for you. That is your job.

1. **Draft** (4 min). Write your own version for your `tasks.md` . Do not run it yet.
2. **Coach and revise** (5 min). Run `/parallel-coach 2` and paste your draft as your next message. Revise until the coach says the prompt is ready.

Disagree with the coach? Say *"run it anyway"*. It will let you, and tell you what to watch for.

**Done when:** the coach says your team prompt is ready, and task 1 is done.

### 3. Start the team and watch it (10 min)

Before you paste, write two predictions on paper. **Will the lead wait for its teammates? Which file will a teammate touch that it should not?** A wrong prediction is fine. Only a missing one is a problem.

1. Paste your prompt into the lead, in terminal 2. The coach does not run this one for you. A team can only start from the session you type in.
2. Press **Ctrl+T**. This opens the shared task list. Within a minute you should see tasks claimed by named teammates. Is the list still empty after a minute? Then Claude used plain subagents. Tell the lead: *"Use an agent team, not subagents."* The agent panel below the prompt input shows subagents too, so the panel alone does not tell you.
3. Use the **up and down arrows** to select a teammate, then press **Enter**. You are now in its transcript. Read what it is doing. To leave, select the lead's row again with the arrows. Do not press **Escape** inside a transcript: that interrupts the teammate. You can type to a teammate here. Only do that when it is stuck.

4. Permission prompts from teammates appear in the **lead's** row, not in the teammate's transcript. If nothing moves for a while, go back to the lead and answer the prompt.
5. Refresh `http://localhost:5173` every minute. Stories appear while you watch.

Watch the lead too. If it starts editing story files itself, tell it: *"Wait for your teammates to finish."*

**Done when:** both teammates report done, every story task in `tasks.md` is ticked, and both stories show in the browser.

## 4. Quality gates (10 min)

Nobody has reviewed this code yet. You run three checks, in the order a reviewer would find problems. Then a command repeats them.

1. **Does it do what the spec says?** (4 min) Open `specs/001-*/quickstart.md` and follow its steps in the browser. Then test the three gaps from Exercise 6, task 1:
  - Go to page 2 of the book list. Does it show different books?
  - Go past the last page. What does an empty page show?
  - Open a book id that does not exist, such as `#/books/9999`. What do you see? Is it the raw error text from the backend?
2. **Did the team respect the file boundaries?** (1 min) In terminal 3:

```
git diff --stat foundation
```

Every changed file should belong to one story. A foundation file in that list means a teammate edited code it did not own.

3. **Does it still depend on the contract only?** (1 min) The constitution, principle II, says the frontend may know the HTTP contract and nothing else. A backend address in the code breaks that rule. One grep:

```
grep -rn "8080\\|localhost" src/ index.html
```

Any hit is a constitution violation.

4. **Write down what you found, then run the check** (3 min). The check is the `/verify-exercise 7` command. Run it and paste your team prompt when it asks. It grades the prompt first. Then it checks the files for the same three things, plus the error paths from principle V. Bring its report to the closing round.

5. **Commit** (1 min):

```
git add -A && git commit -m "feat: bookstore-web, built by agent team"
```

**Done when:** you followed the quickstart, ran the two commands, and `/verify-exercise 7` has run.

---

## Closing round (5 min)

---

The trainer asks people at random. Have answers ready:

- What did the coach flag in your team prompt before it said the prompt was ready?
  - Did the lead wait, or did it start building a story itself? Did your prediction from task 3 come true?
  - Did a teammate touch a file it did not own? Which one, and why?
  - In Exercise 6 you marked some answers as guesses. Find one in the running frontend. Does the code treat it as a fact?
  - Close with **one take-away**: which part of this exercise needed a team, and which part would have been faster in a single session?
- 

## What you should have

---

|                              |                                        |
|------------------------------|----------------------------------------|
| bookstore-web/               |                                        |
| index.html, styles.css, src/ | built from your spec                   |
| specs/001-*/tasks.md         | every story task ticked                |
| git log                      | "spec" → "foundation" → "feat" commits |

Every line of application code in this project was written from a spec you wrote. You typed none of it yourself.

**Before session 8:** bring a project of your own that builds and has tests, cloned on your laptop. Exercise 8 runs on it.